

// career roadmap — 2026

Path to Adobe Frontend Engineer

React.js Role · R165379 · Noida, UP | 10–12 months to interview confidence | Starting advantage: You already have React ✓

7	10–12	3	2h	4
PHASES	MONTHS	PROJECTS	DAILY	INT. ROUNDS

★ **YOUR EDGE** You're not starting from zero. You bring **React + Ionic, Electron, real product experience**, and familiarity with **performance-heavy systems**. Most candidates only have CRUD apps — your background in GPU/video and control systems is genuinely rare. Package it right and Adobe will notice.

// LEARNING PHASES

P1

JavaScript Mastery

The foundation everything else is built on

1.5 – 2 months

■ Closures & Scope	■ Event Loop	■ Promises + async/await
■ Execution Context	■ Prototypal Inheritance	■ this binding
■ Debounce / Throttle	■ Memory Leaks	■ DOM Internals
■ Browser Rendering Pipeline	■ ES6+ Features	■ Modules (ESM/CJS)
■ Functional Programming	■ Web APIs	

→ [javascript.info](#) — best JS learning resource

→ [MDN Web Docs](#) — deep browser APIs reference

P2

Advanced React

Go from using React to understanding React

2 months

■ Reconciliation Algorithm	■ Virtual DOM Diffing	■ Fiber Architecture
■ Render & Commit Phase	■ Why re-renders happen	■ Batching updates
■ useState internals	■ useEffect lifecycle	■ useMemo/useCallback WHY
■ useRef vs useState	■ Custom hooks patterns	■ Context API limits
■ Code Splitting	■ Lazy Loading	■ Suspense & Error Boundaries

■ React Query / TanStack	■ Redux Toolkit	■ Zustand
■ Accessibility		

→ [react.dev](#) — official deep docs
 → [epicreact.dev](#) — Kent C. Dodds advanced patterns

P3

TypeScript

Adobe explicitly requires it — don't skip

3 – 4 weeks

■ Types vs Interfaces	■ Generics	■ Utility Types (Partial, Pick, Omit)
■ Type Narrowing	■ Discriminated Unions	■ React + TS Patterns
■ API Response Typing	■ tsconfig basics	

→ [Convert 2 old React projects to TypeScript](#)
 → [Type a REST API client fully](#)

P4

Frontend System Design

What separates engineers from developers

1 – 1.5 months

■ Scalable folder structures	■ Design systems	■ Component architecture
■ State architecture patterns	■ Caching strategies	■ Authentication flows
■ CSR vs SSR vs SSG	■ Hydration	■ Microfrontends basics
■ CDN & asset optimization	■ Browser storage	■ WebSockets
■ Security basics (XSS, CSRF)	■ Testing (Jest, RTL)	

→ [nextjs.org/docs](#) — [learn SSR/SSG in practice](#)

P5

DSA for Interviews

Ongoing daily — Adobe WILL test this

Ongoing (daily)

■ Arrays & Strings	■ HashMaps	■ Sliding Window
■ Two Pointers	■ Stack & Queue	■ Linked Lists
■ Binary Search	■ Trees (BFS/DFS)	■ Recursion
■ Graph basics	■ DP basics (Memoization)	

→ [leetcode.com](#) — filter by "Adobe" tagged problems
 → [2 problems/day](#), focus on Medium difficulty

P6

Flagship Projects

Make your resume look like product-company work

Parallel to P2–P4

■ PDF/Document Web App (like Acrobat) — PDF.js, React, TS, Annotations, Search

■ Real-Time Collaboration App — WebSockets, Zustand, Presence, Comments

■ Video/Performance Dashboard — Electron background, live feeds, metrics UI

→ [PDF.js](#) — [Mozilla's PDF renderer](#)

→ [socket.io](#) — [WebSocket library](#)

P7

AI Integration Skills

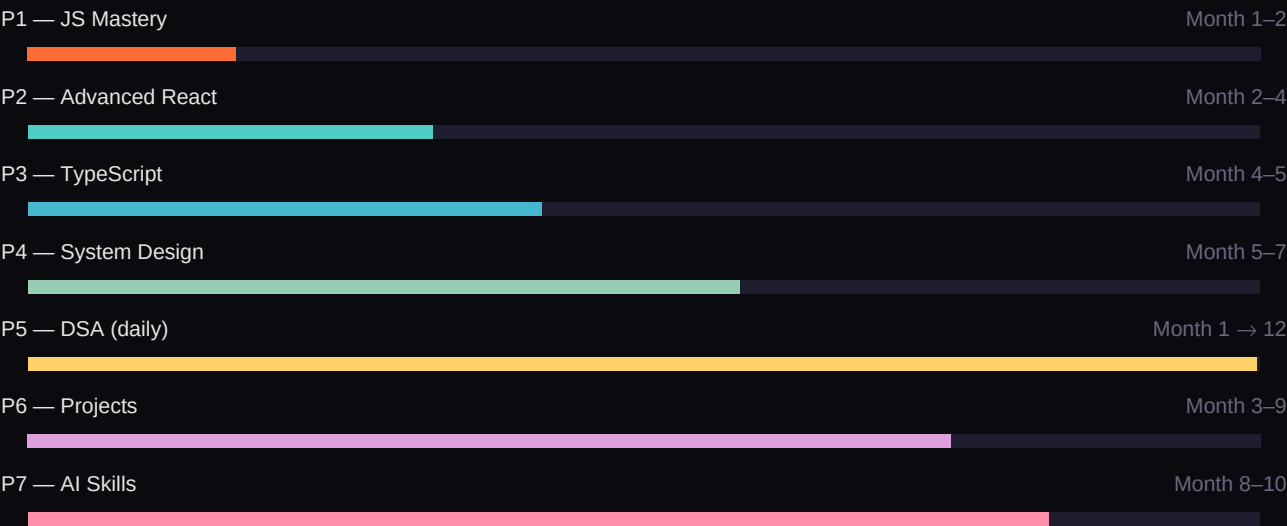
JD explicitly mentioned this — don't ignore

2 – 3 weeks

■ Prompt Engineering basics	■ Embeddings (concept)	■ RAG basics
■ Vector DB basics	■ Semantic search in UI	■ Evaluating AI outputs

→ [platform.openai.com/docs](#)
→ [langchain.com](#)

// TIMELINE AT A GLANCE



// YOUR DAILY SCHEDULE

Weekdays (after office)		Weekends	
Office ends	~8:00 PM	Build flagship projects	3 – 4 hrs
Wind down / dinner	8:00 – 8:45 PM	Mock interviews	1 hr
DSA — 2 problems	8:45 – 9:45 PM	Read engineering blogs	30 min
React / System Design / Projects	9:45 – 10:45 PM	Resume / LinkedIn updates	30 min

// ADOBE INTERVIEW STRUCTURE

R1 DSA + JavaScript Arrays, strings, HashMaps Closures, event loop, async JS output-based questions	R2 React Deep Dive Reconciliation explanation Why useMemo? When not to? useRef vs useState Optimize slow component	R3 Machine Coding Build: Todo, Data Table, File Explorer, Search UI, Kanban — clean arch + perf	R4 System Design Frontend architecture Scalability decisions State & caching strategy Performance trade-offs
--	---	--	---

// RESUME LANGUAGE THAT GETS NOTICED

■ Built real-time collaborative systems	■ Optimized React rendering performance
■ Implemented scalable frontend architecture	■ TypeScript-first component development
■ Built document-based interfaces with PDF.js	■ Reduced bundle size / improved LCP
■ Led code reviews in Agile team	■ Integrated AI features into frontend apps